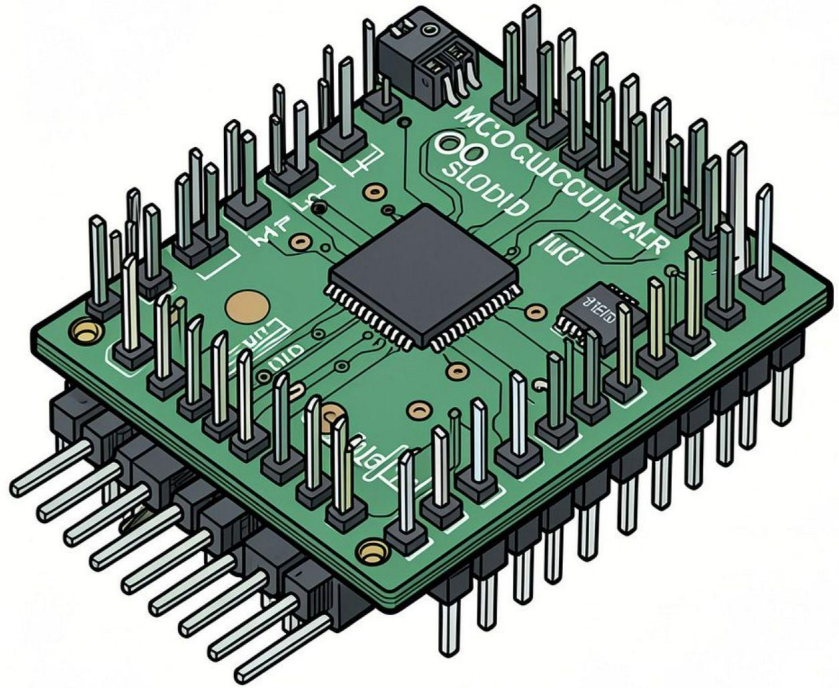
GPIO

Course: Microcontroller and Interfacing

Group: 15(GPIO)

Group Members:

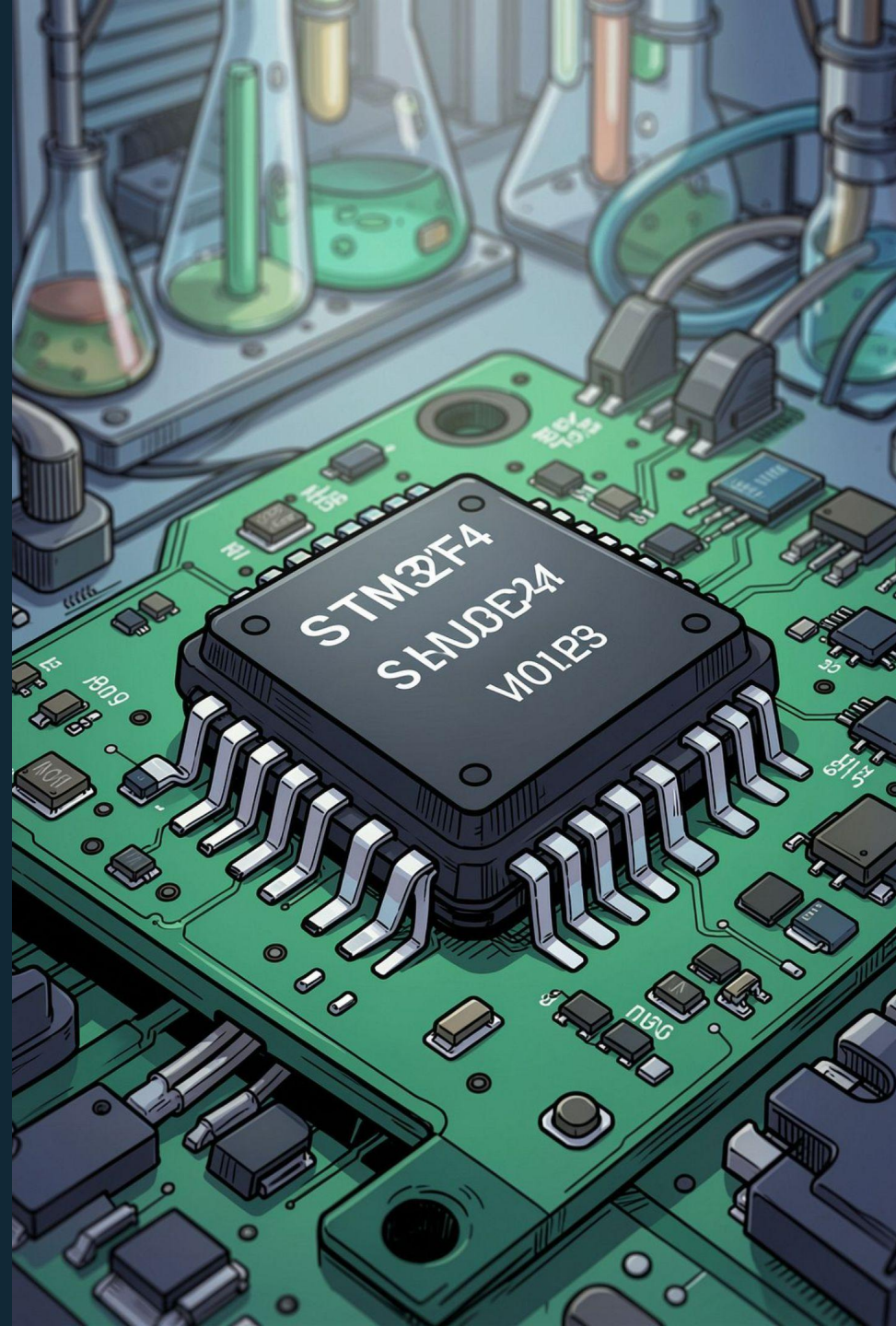
1. Masud Rana (2211176104)
2. Samudra Mazumder (2211176134)
3. Sumayta Rahman (2212276116)



GPIO Architecture of STM32F446

GPIO (General-Purpose Input/Output) is the configurable digital interface that lets the STM32F446 CPU and peripherals interact with external circuits. A single physical pin can be configured to sample signals, drive outputs, or hand control to an internal peripheral — making GPIO both flexible and central to embedded design.

This deck explains the STM32F446 GPIO signal path, register-level controls, electrical options (pulls, output type, speed), alternate functions, atomic set/reset, and practical patterns for reliable interfacing and interrupt-driven designs.



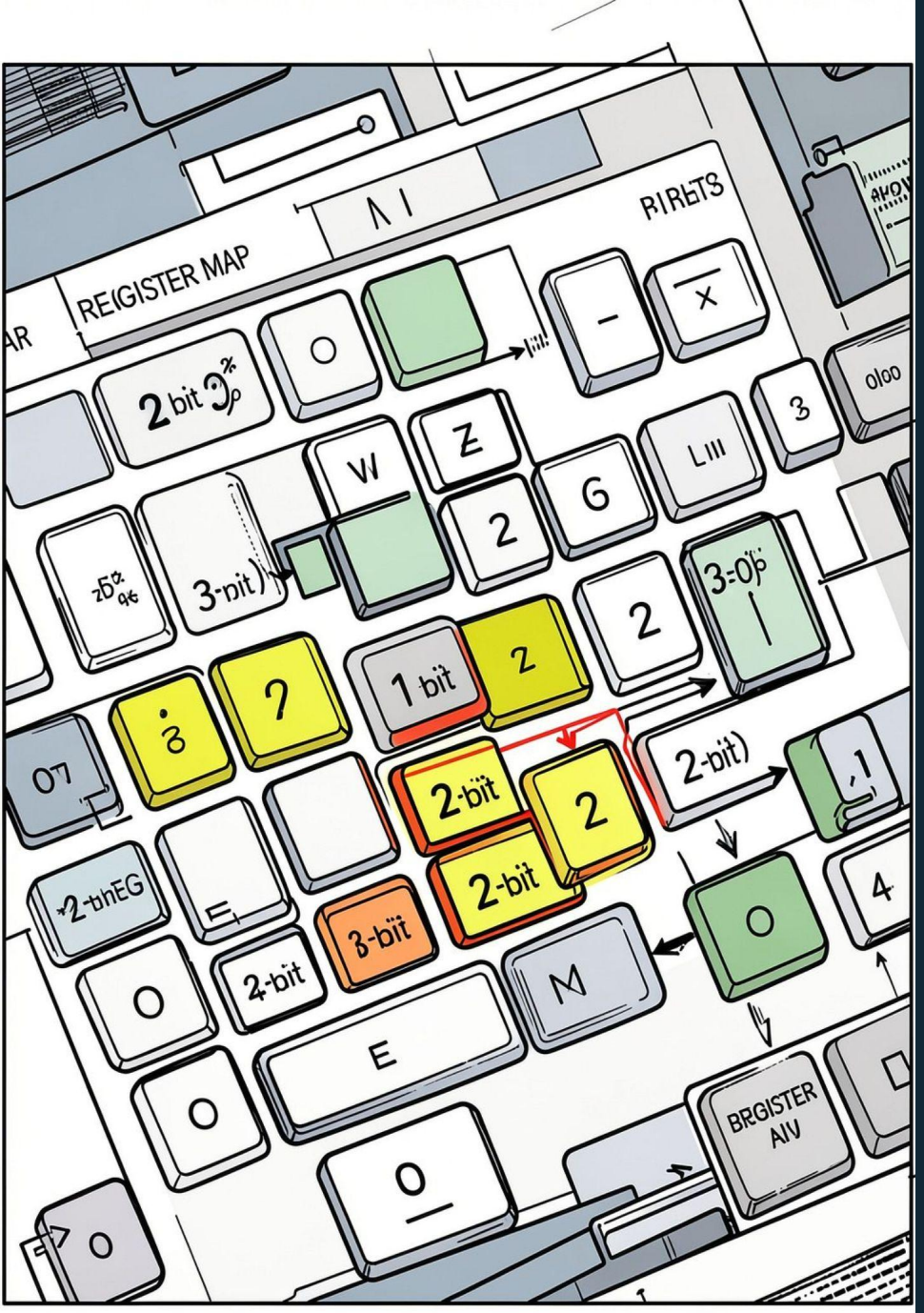
STM32 GPIO Signal Path — Conceptual Overview

The GPIO signal path has two complementary flows that meet at the physical pin. On the output side, software writes to the GPIO output data path, which is then translated by the output driver into a logic-level voltage or current on the pad. On the input side, the external world drives the pin, and the signal is first conditioned by the pad circuitry before being sampled by the input buffer and made visible to the CPU through the input data register.

Output path: CPU write → ODR/BSRR → output latch → output driver → pin pad → external device

Input path: external signal → pin pad → protection network → input buffer / Schmitt trigger → IDR → CPU read

The output path is usually controlled through the output data register (ODR) or the bit set/reset register (BSRR). BSRR is preferred for atomic pin updates because it avoids read-modify-write hazards and lets firmware set and reset different pins in a single write. The output driver itself is responsible for sourcing or sinking current, so its behavior depends on whether the pin is configured as push-pull or open-drain, and whether the output speed setting is optimized for low-noise transitions or faster edges.



GPIO Modes and the MODER Register

Each pin's mode is encoded in MODER with two bits: MODER[2n+1:2n]. The four modes:

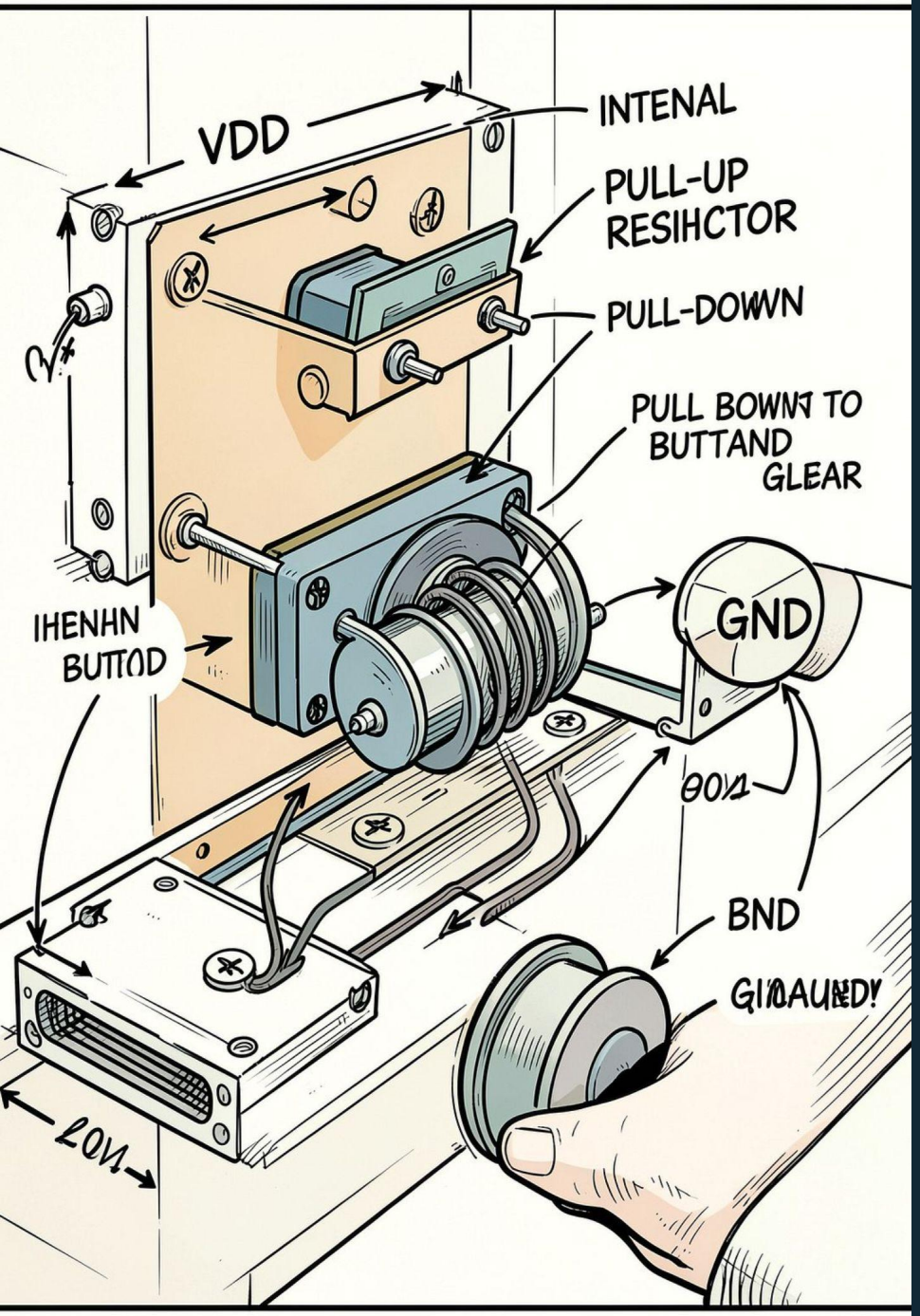
- 00 — Input: digital input, input buffer active
- 01 — General Purpose Output: ODR/BSRR control, output driver enabled
- 10 — Alternate Function: peripheral drives the pin via AFR
- 11 — Analog: analog/low-power (input buffer disabled)

Example: PA5 uses MODER[11:10]. After reset pins default to safe states (typically input with pull-down/up determined by hardware) to avoid driving unknown loads on connected circuits.

GPIO Inputs — IDR and Electrical Considerations

When configured as input, the pin's voltage passes through the protection network and input buffer before being sampled by the IDR register. IDR[n] reads 0 = LOW and 1 = HIGH. Inputs do not drive the line, so the external circuit must establish a valid logic level.

GPIO input pins are designed to present very high input impedance, which minimizes DC loading on the source signal. Even so, the pin and its package trace add capacitance, so fast edges can be slowed and noisy or high-impedance sources may be more vulnerable to distortion. For reliable operation, keep source impedance low enough to charge the input capacitance quickly, and use buffering, series resistance, or RC filtering when the signal needs conditioning before it reaches the MCU.



Pull-Up / Pull-Down — PUPDR Register

Each pin has two PUPDR bits (PUPDR[2n+1:2n]) with values:

- 00 — No pull
- 01 — Pull-up (weak)
- 10 — Pull-down (weak)
- 11 — Reserved

Internal pulls provide a weak bias to define default logic states, reducing BOM by avoiding external resistors for simple inputs. Choose pull direction based on your external circuit (e.g., pull-up for active-low buttons to GND).

GPIO Output: ODR, BSRR, and Atomic Control

Output state is stored in ODR (ODR[n] = 1 → HIGH, 0 → LOW). For atomic single-pin writes, use BSRR:

- BSRR[0:15] — set pins corresponding to bits = 1
- BSRR[16:31] — reset pins when bits = 1 (write-only)

Example register usage:

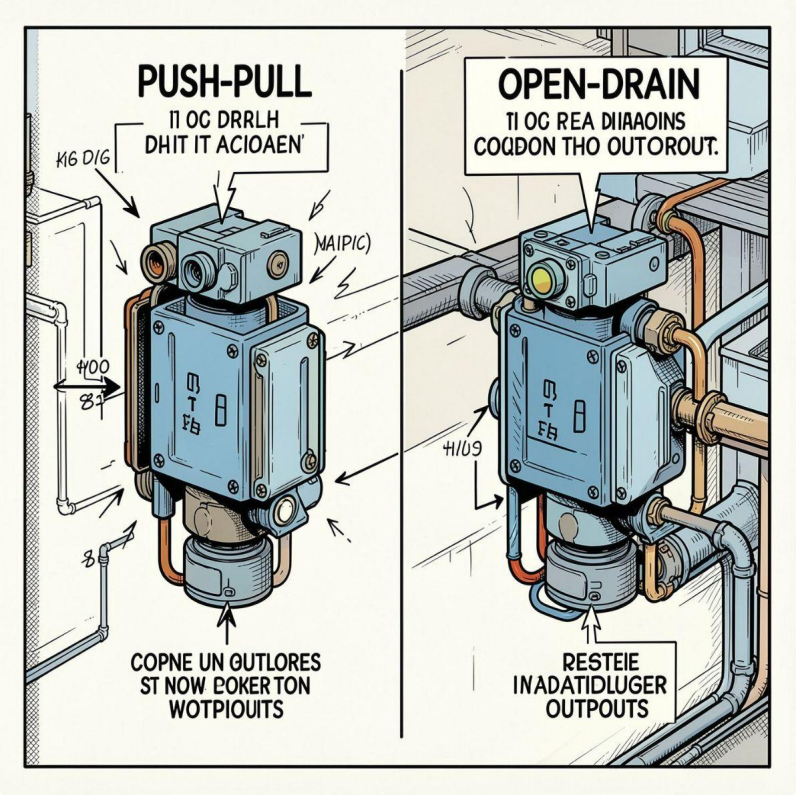
```
GPIOA->BSRR = GPIO_PIN_5;           // set PA5
GPIOA->BSRR = GPIO_PIN_5 << 16;     // reset PA5
```

BSRR avoids read-modify-write races, enabling safe concurrent access from ISRs and multi-master subsystems.

Output Type, Speed, and Electrical Tradeoffs

Output Type

OTYPER: Push-Pull vs Open-Drain



Output Speed — OSPEEDR

OSPEEDR configures rise/fall rates. Higher speed reduces edge time but increases switching current, EMI, and ringing. Principle: use the lowest speed that meets timing and signal-integrity requirements.

- Push-pull: actively drives both HIGH and LOW — good for LEDs, logic signals
- Open-drain: drives LOW only; requires pull-up for HIGH — essential for shared-bus protocols like I²C

Alternate Functions and the AFR Registers

Switching a pin to a peripheral requires two steps:

1. Set MODER to Alternate Function (10)
2. Choose the AF index in AFRL (pins 0–7) or AFRH (pins 8–15); each AF field is 4 bits (AFR[4n+3:4n])

Example: PA5 → set MODER bits for AF, then program AFRL/AFRH to select the specific peripheral function available on PA5 for your package. Always cross-check the STM32F446 datasheet and alternate-function matrix for valid mappings.

Alternate functions are the STM32's pin-routing layer: they connect a physical pad to an on-chip peripheral such as USART, SPI, I²C, timers, or external interrupt lines. Because multiple peripherals may be able to use the same pin, the AFR selection determines which internal signal is actually driven or sampled. This makes pin planning an essential part of board design, especially when a project needs several serial interfaces, PWM outputs, or capture inputs at the same time.

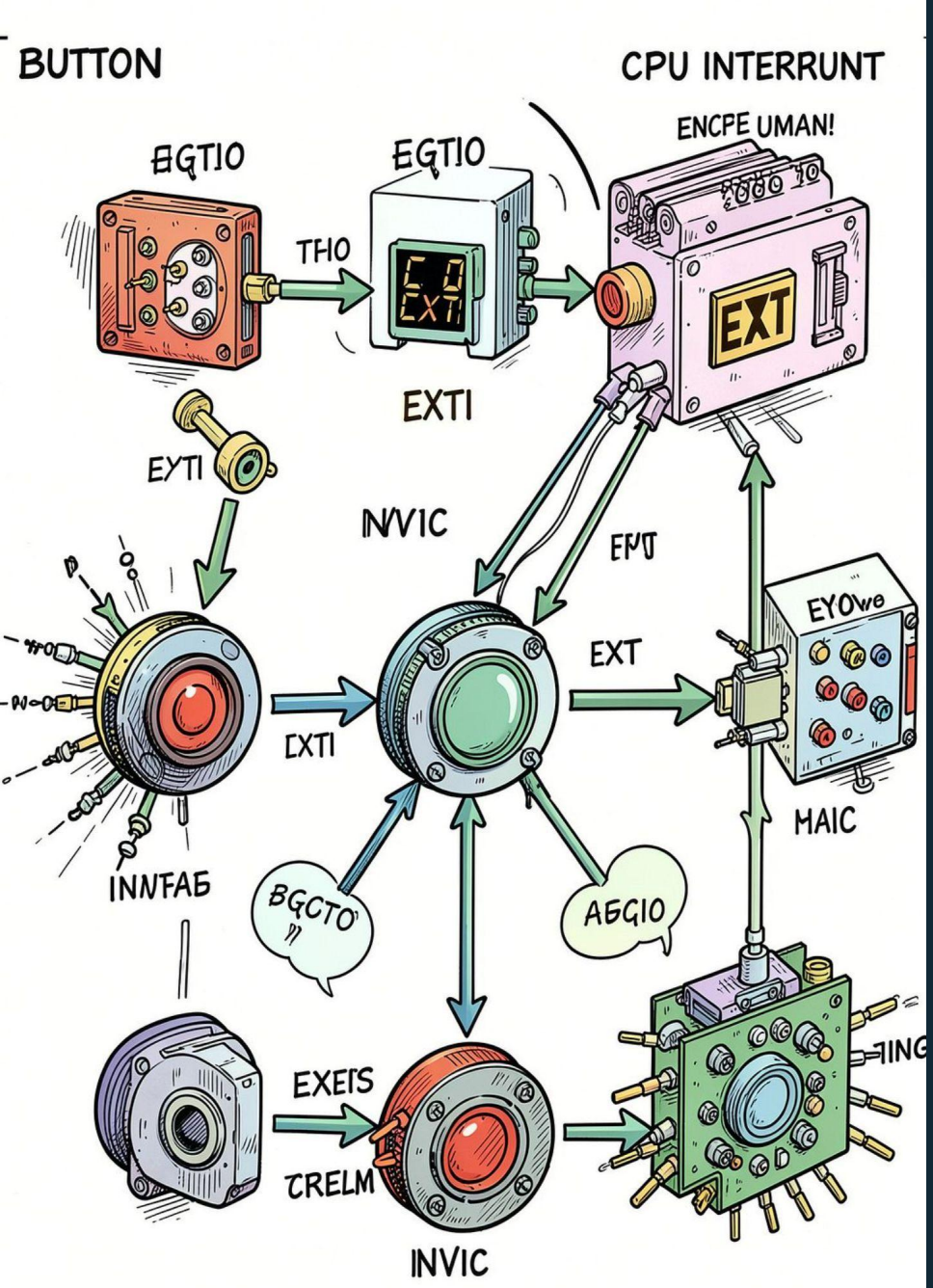
Complete GPIO Configuration Sequence (Register-Level)

Typical initialization for PA5 as a push-pull digital output (register-level):

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;        // enable GPIOA clock
GPIOA->MODER &= ~(3U << (5*2));
GPIOA->MODER |=  (1U << (5*2));             // MODER = 01 (output)
GPIOA->OTYPER &= ~(1U << 5);                // push-pull
GPIOA->OSPEEDR &= ~(3U << (5*2));           // default/low speed
GPIOA->PUPDR &= ~(3U << (5*2));             // no pull
GPIOA->BSRR = GPIO_PIN_5;                   // set PA5
```

HAL abstraction example:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```



GPIO Interrupts (EXTI) and Practical Interfacing

EXTI connects GPIO lines to the interrupt controller so inputs can trigger CPU interrupts instead of polling. Typical flow: Button press → GPIO input → EXTI triggers → NVIC delivers IRQ → ISR handles event (debounce, state change, toggle LED).

Efficiency

Interrupts eliminate constant polling, reducing CPU load and power consumption.

Responsiveness

Faster reaction to asynchronous events (buttons, sensors, external devices).

Determinism

Atomic BSRR writes plus careful ISR design avoid race conditions in concurrent systems.

Thank You